Homework 3

Matthew Wilkinson

mmw071@uark.edu

100434077

1. Describe the purpose of the homework and what you did / what you implemented.
2. Describe your approach and explain how you solved the problem (the key ideas for implementing each required function).
3. Discuss any bugs or problems you encountered and how you fixed them.
4. Include screenshots or images of your results.

Purpose

The purpose of the homework is for students to implement tree data structures, specifically a Binary Search Tree (BST) and an AVL Tree. The goal was to understand how trees store data efficiently and how operations like insert, find, and remove work. For the BST, I implemented basic operations including insert, remove, find, findMinimum, findMaximum, and clear. For the AVL tree, I extended the BST by adding balancing logic. This included implementing rotations (left and right) and making sure the tree stays balanced after every insert and remove.

Approach

For the BST, I followed the recursive structure from lecture. The insert function compares the key and moves left or right until it finds a null position to insert the new node. The find function iteratively searches the tree by comparing keys and moving left or right. For findMinimum and findMaximum, I simply traversed left or right until no further child exists. The remove function handled the three standard cases: deleting a leaf node, deleting a node with one child, and deleting a node with two children by replacing it with the minimum value from the right subtree. The clear function used a post-order traversal to delete all nodes safely.

For the AVL tree, I extended the BST logic by maintaining balance after every insert and remove. After each operation, I updated the node height and calculated the balance factor. Depending on the balance, I applied the correct rotation (left, right, left-right, or right-left) to restore balance. This ensures the tree remains efficient with a height of O(log n).

Bugs/Problems

One problem I had was with the main.cpp file. It had an error saying 'numeric_limits' is not a member of std. This occurred because the file used *std::numeric_limits<std::streamsize>::max()* in main.cpp and the library wasn't included in the file. So, I fixed this by including the library limits. I inserted the line '#include <limits>' on line 13 of main.cpp which seemed to resolve the issue. I know I am not supposed to edit main.cpp but I couldn't figure out any other simple way to resolve the problem. I also had a similar issue and had to include '#include <sstream>' in main.cpp as well.

I still had problems with OpenCV working with my machine. I think I just need to restart, I guess. However, my makefile still worked perfectly and the programs executed like I expected from the homework 3 pdf. Besides that, I really didn't have issues with this assignment, the lecture notes helped a lot for BST. Had some struggles getting AVL to work fully, as expected.

RESULTS SECTION ON NEXT PAGE

Results

Terminal of Mingw64 of Final Results:

```
/c/users/mww61/gitRepos/algor/algorithmsclass/hw3
mww61@Matthew MINGW64 /c/users/mww61/gitRepos/algor/algorithmsclass/hw3
$ mingw32-make
g++ -I./include/ -std=c++14  src/linked_list.cpp src/graph.cpp src/queue.cpp src/stack.cpp src/bfs.
pp src/bst.cpp src/avl.cpp src/main.cpp  -o bin/main
./bin/main
Perform unit test on your BST implementation
Insert 0 into BST
Find 0. Found 0 in BST
Find 1. Not found 1 in BST

Insert 1 into BST
Find 1. Found 1 in BST
Find 2. Not found 2 in BST

Insert 2 into BST
Find 2. Found 2 in BST
Find 3. Not found 3 in BST

Insert 3 into BST
Find 3. Found 3 in BST
Find 4. Not found 4 in BST

Insert 4 into BST
Find 4. Found 4 in BST
Find 5. Not found 5 in BST

Insert 5 into BST
Find 5. Found 5 in BST
Find 6. Not found 6 in BST

Insert 6 into BST
Find 6. Found 6 in BST
Find 7. Not found 7 in BST

Insert 7 into BST
Find 7. Found 7 in BST
Find 8. Not found 8 in BST

Insert 8 into BST
Find 8. Found 8 in BST
Find 9. Not found 9 in BST

Insert 9 into BST
Find 9. Found 9 in BST
Find 10. Not found 10 in BST

Maximum value in BST: 9

Remove 0 out of BST
Find 0. Not found 0 in BST
Find 1. Found 1 in BST

Remove 1 out of BST
Find 1. Not found 1 in BST
Find 2. Found 2 in BST

Remove 2 out of BST
Find 2. Not found 2 in BST
Find 3. Found 3 in BST

Remove 3 out of BST
Find 3. Not found 3 in BST
Find 4. Found 4 in BST

Remove 4 out of BST
Find 4. Not found 4 in BST
Find 5. Found 5 in BST

Remove 5 out of BST
Find 5. Not found 5 in BST
Find 6. Found 6 in BST

Remove 6 out of BST
Find 6. Not found 6 in BST
Find 7. Found 7 in BST
```

Terminal of Mingw64 of Final Results (Continued…):

```
/c/users/mww61/gitRepos/algor/algorithmsclass/hw3

Remove 6 out of BST
Find 6. Not found 6 in BST
Find 7. Found 7 in BST

Remove 7 out of BST
Find 7. Not found 7 in BST
Find 8. Found 8 in BST

Remove 8 out of BST
Find 8. Not found 8 in BST
Find 9. Found 9 in BST

Your BST implementation is correct
Perform unit test on your AVL implementation
--- Starting AVL Unit Test ---
Insert 0 into AVL
Current Tree Height (at root): 0
Find 0. Found 0 in AVL

Insert 1 into AVL
Current Tree Height (at root): 1
Find 1. Found 1 in AVL

Insert 2 into AVL
Current Tree Height (at root): 1
Find 2. Found 2 in AVL

Insert 3 into AVL
Current Tree Height (at root): 2
Find 3. Found 3 in AVL

Insert 4 into AVL
Current Tree Height (at root): 2
Find 4. Found 4 in AVL

Insert 5 into AVL
Current Tree Height (at root): 2
Find 5. Found 5 in AVL

Insert 6 into AVL
Current Tree Height (at root): 2
Find 6. Found 6 in AVL

Insert 7 into AVL
Current Tree Height (at root): 3
Find 7. Found 7 in AVL

Insert 8 into AVL
Current Tree Height (at root): 3
Find 8. Found 8 in AVL

Insert 9 into AVL
Current Tree Height (at root): 3
Find 9. Found 9 in AVL

Maximum value in AVL: 9

Remove 0 out of AVL
Find 0. Not found 0 in AVL
Find 1. Found 1 in AVL
Tree height after removal: 3

Remove 1 out of AVL
Find 1. Not found 1 in AVL
Find 2. Found 2 in AVL
Tree height after removal: 3

Remove 2 out of AVL
Find 2. Not found 2 in AVL
Find 3. Found 3 in AVL
Tree height after removal: 3

Remove 3 out of AVL
Find 3. Not found 3 in AVL
Find 4. Found 4 in AVL
Tree height after removal: 2
```

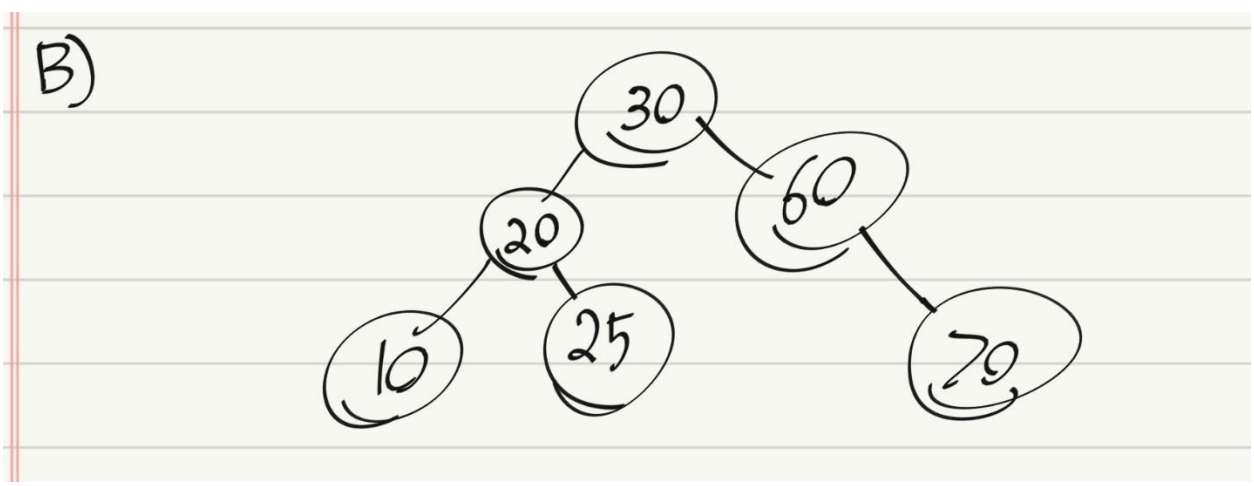
Terminal of Mingw64 of Final Results (Continued…):

```
Remove 3 out of AVL
Find 3. Not found 3 in AVL
Find 4. Found 4 in AVL
Tree height after removal: 2

Remove 4 out of AVL
Find 4. Not found 4 in AVL
Find 5. Found 5 in AVL
Tree height after removal: 2

Remove 5 out of AVL
Find 5. Not found 5 in AVL
Find 6. Found 6 in AVL
Tree height after removal: 2

Remove 6 out of AVL
Find 6. Not found 6 in AVL
Find 7. Found 7 in AVL
Tree height after removal: 1

Remove 7 out of AVL
Find 7. Not found 7 in AVL
Find 8. Found 8 in AVL
Tree height after removal: 1

Remove 8 out of AVL
Find 8. Not found 8 in AVL
Find 9. Found 9 in AVL
Tree height after removal: 0

Your AVL implementation is correct


Shortest path from JBHT to HAPG: JBHT -> HILL -> WJWH -> HAPG
Total Distance: 47354
You have to use OpenCV to visualize your map road


mww61@Matthew MINGW64 /c/users/mww61/gitRepos/algor/algorithmsclass/hw3
```

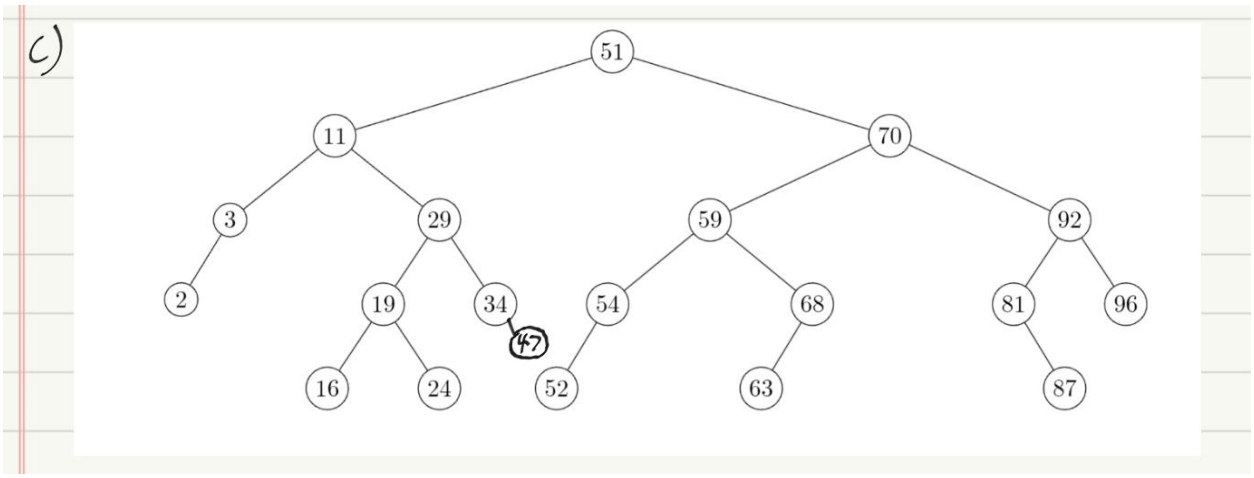
Tree Practice Problem Results

A. The maximum number of nodes of a binary tree of height *h* is: $\mathbf{2^{h+1} - 1}$

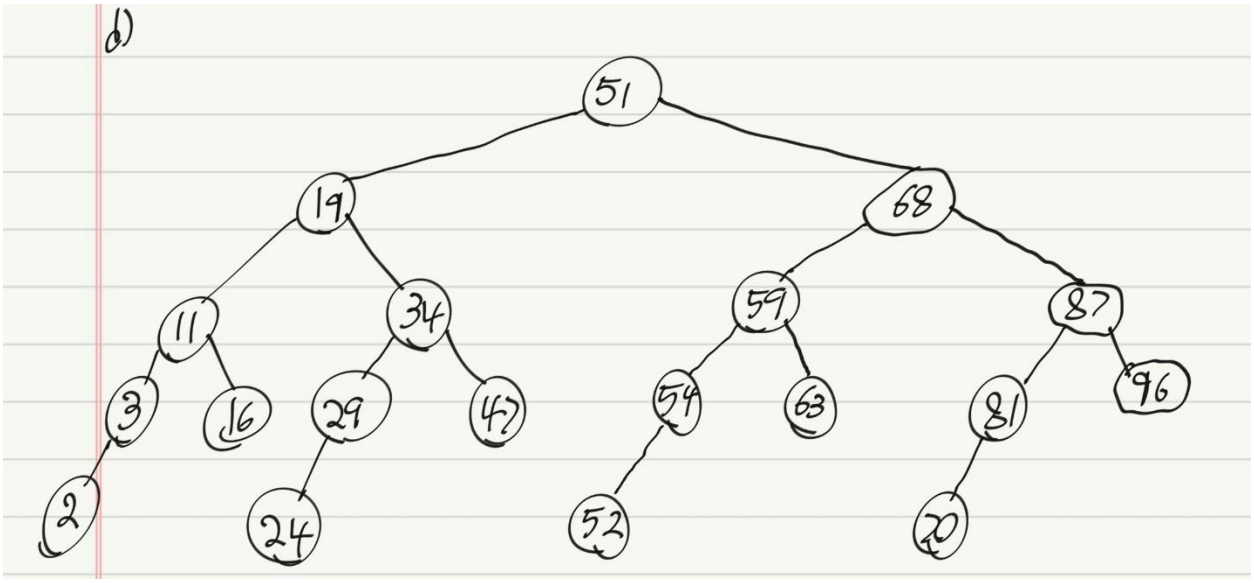
B.



C.



D.



E. **O(log n)**

This is because the tree stays balanced, so its height is logarithmic. Finding the correct place to insert takes **O(log n)**, and the rotations used to rebalance only take constant time.

F. **O(log n)**

This is because finding the node takes **O(log n)**, and any rebalancing also only happens along the path of height **O(log n)**.